

Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Лабораторна робота № 1

з дисципліни «Системне програмування вбудованих систем»
на тему «Налаштування середовища розробки та запуск Lockbox у Wokwi»

Виконав:

студент групи ІО-32

Омельченко Богдан Юрійович

Перевірив:

Гордієнко Нікіта Юрійович

Мета роботи:

Я хотів підготувати проєкт так, щоб прошивку можна було зібрати в однаковому середовищі та запустити без фізичної плати. Перший практичний результат для цієї лабораторної: після запуску у Wokwi програма має вивести повідомлення в UART і перемикає світлодіод. Це дає змогу перевірити збірку, налаштування тактування та роботу двох простих виходів, перш ніж підключати датчики.

Роботу виконано індивідуально за формою Б. Збірку й демонстрацію проводив на Ubuntu 24 у віртуальній машині; STM32CubeMX використовував для перегляду конфігурації мікроконтролера. Фізичної плати в цій роботі немає.

Постановка завдання:

Тема подальшого проєкту **Lockbox**: контроль шафки з речами. Ідея полягає в тому, щоб фіксувати струс корпусу, зміну освітленості всередині й рух поруч. Для першої лабораторної достатньо підготувати основу: репозиторій, опис вимог, збірку STM32 HAL, конфігурацію в CubeMX, UART-журнал і блимання LED.

Таблиця 1 – План компонентів Lockbox

Компонент	Інтерфейс	Призначення та стан на кінець ЛР1
MPU6050	I ² C	Виявлення струсу; драйвер і перевірку залишено для ЛР2.
Фоторезистор	ADC	Контроль освітленості всередині шафки; канал і поріг ще не налаштовано.
PIR-датчик	GPIO	Реєстрація руху поруч; у поточній схемі не підключений.
LED та UART	GPIO, USART2	Працюють у першій версії прошивки; це перевірка середовища.

Детальні вимоги й перевірки записано в PRD проєкту. Огляд компонентів є в hardware-components.md. У методичці наведено NUCLEO-F411RE/F446RE. Для симуляції обрано NUCLEO-C031C6, яка є в Wokwi; цю заміну ще потрібно погодити з викладачем.

Хід роботи:

Підготовка проєкту:

Спочатку визначив тему й записав вимоги. Потім створив прошивку на STM32 HAL, кореневий Makefile, Dockerfile і схему Wokwi. Ці файли з'явилися в коміті c0dc0dc;

вимоги й огляд компонентів додані в c18с6а2. Для задачі робота ведеться в гілці feature/lab01-setup від main. Репозиторій: <https://github.com/lilbodyn/Lockbox>.

Обрана конфігурація містить USART2 на PA2/PA3, LED на PA5 і I²C1 на PB8/PB9. I²C1 ініціалізується, але датчик до нього ще не підключений. На рисунку 1 видно розведення в CubeMX.

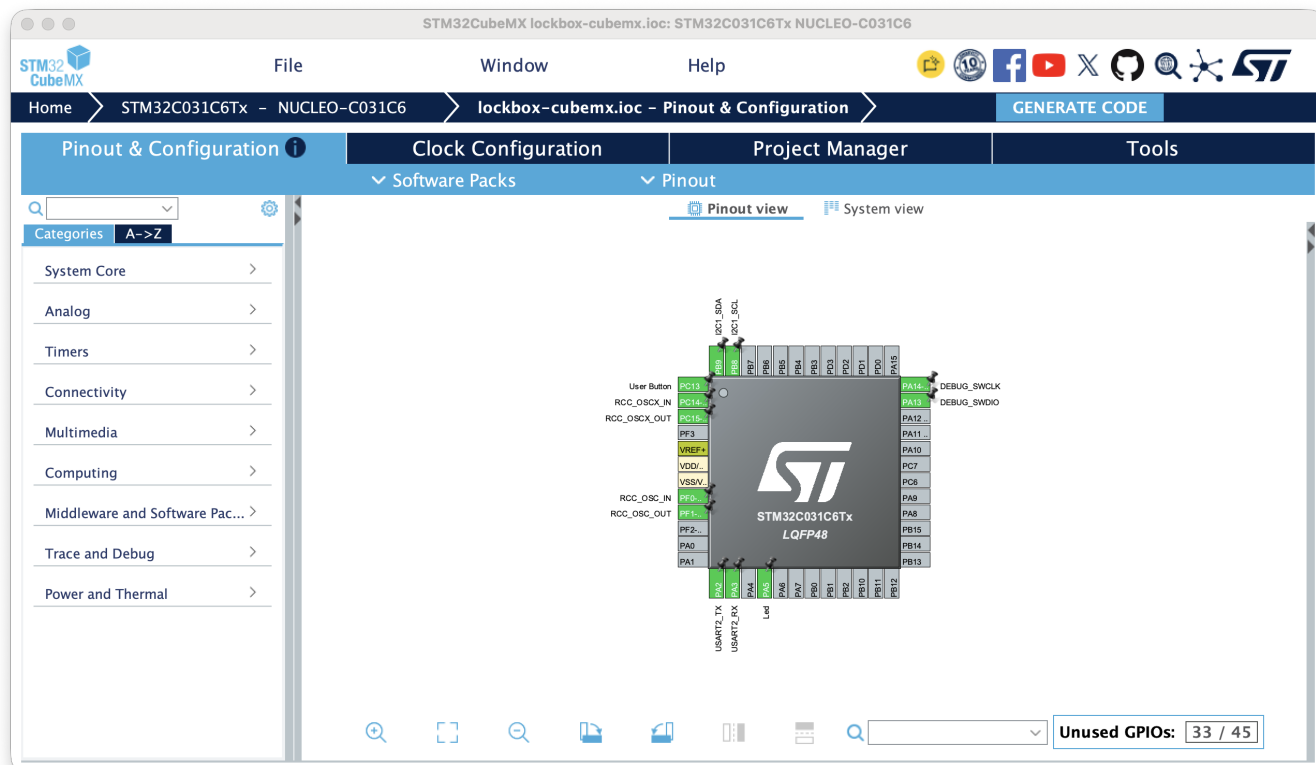


Рисунок 1 – Pinout мікроконтролера STM32C031C6 у CubeMX

USART2 працює в асинхронному режимі зі швидкістю 115200 біт/с, вісьмома бітами даних, без парності й з одним стоп-бітом. На рисунку 2 показані ці параметри. Джерело системної частоти на рисунку 3 дає SYSCLK і HCLK по 48 МГц.

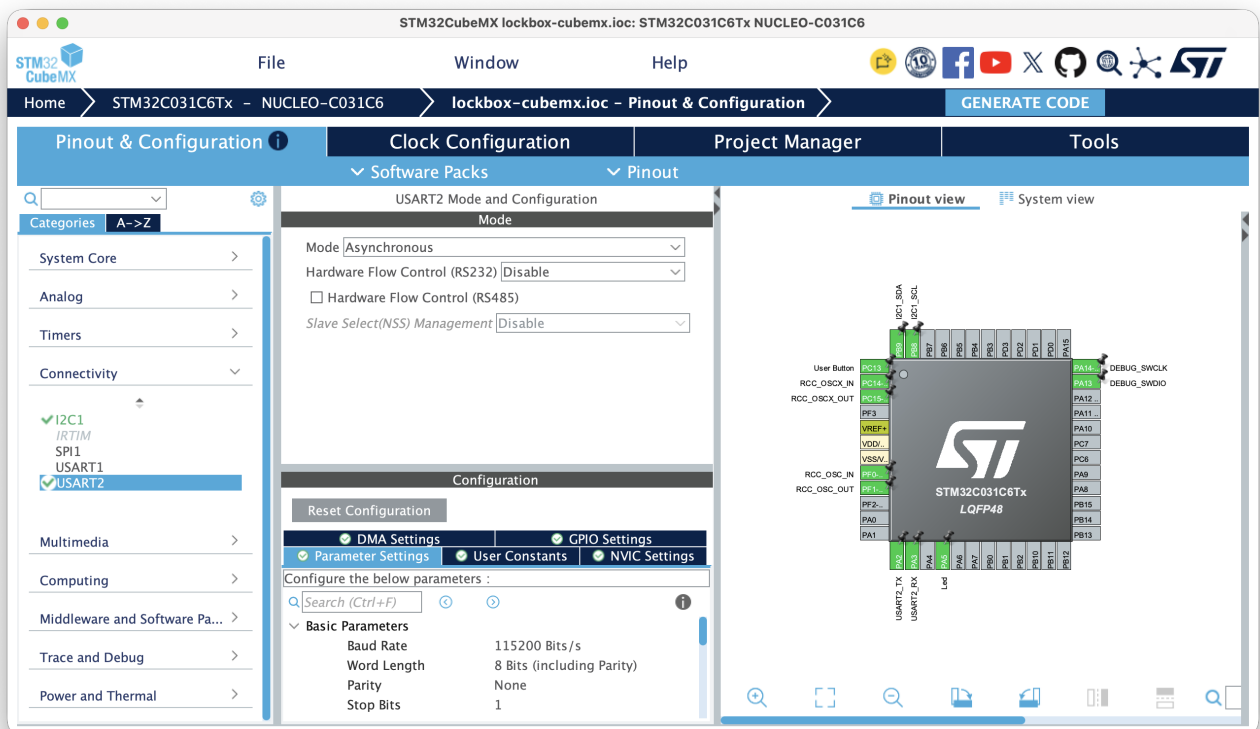


Рисунок 2 – Параметри USART2 у CubeMX

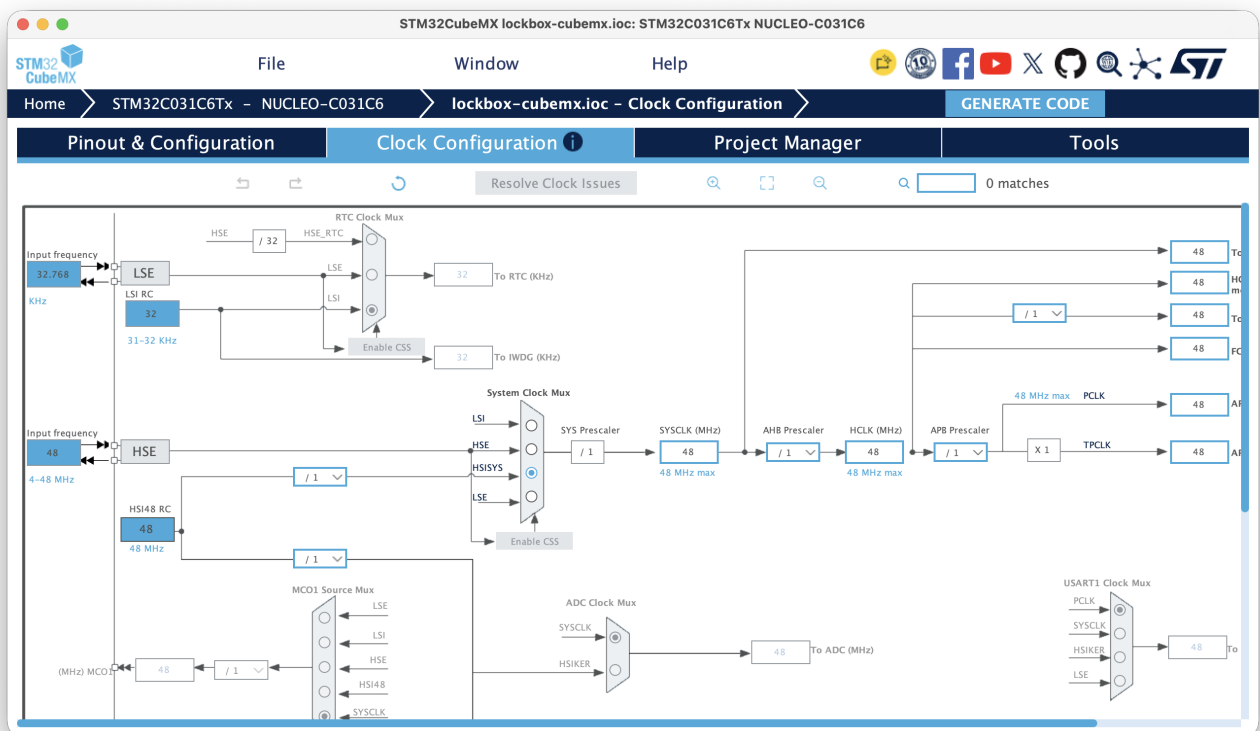
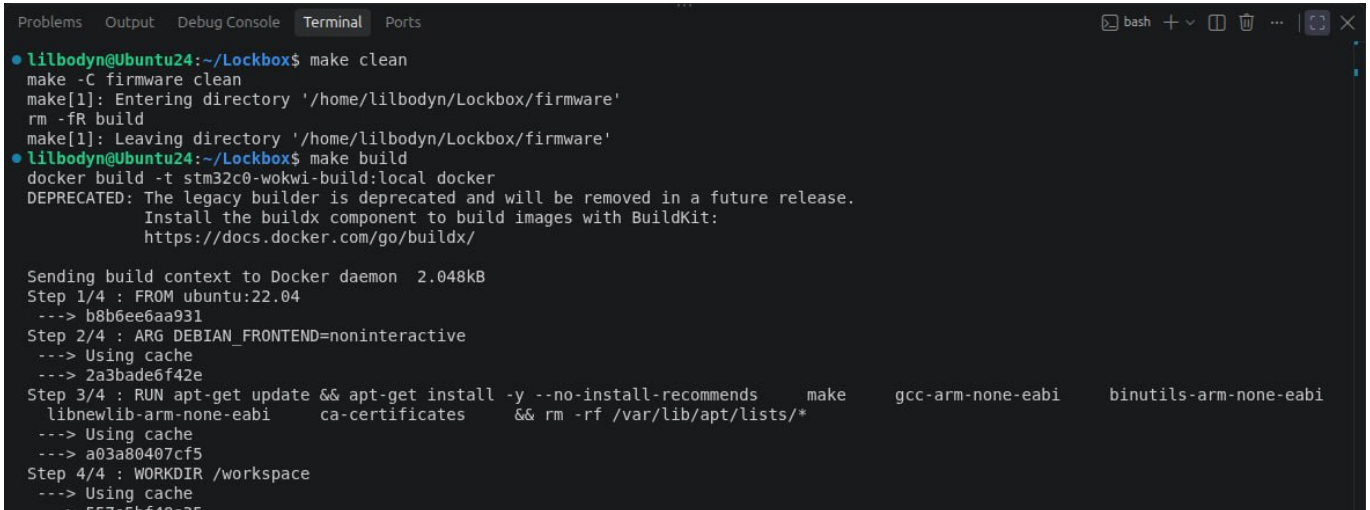


Рисунок 3 – Конфігурація тактування, SYSCLK 48 МГц

Збірка прошивки:

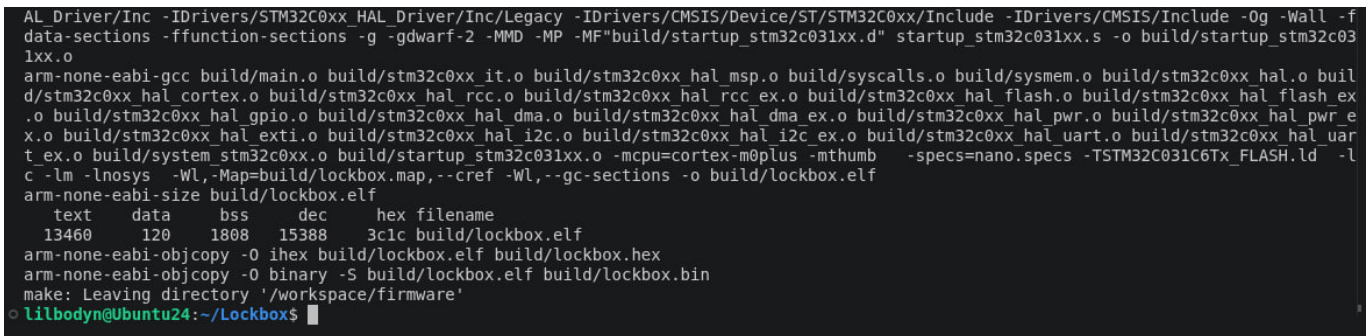
У корені репозиторію виконав `make clean`, потім `make build`. Другу команду Makefile передає контейнеру Docker, у якому встановлений ARM toolchain. Після компіляції отримав `lockbox.elf`, `lockbox.hex` і `lockbox.bin`. Початок збірки, її завершення та файли показано на рисунках 4–6.



```
lilbodyn@Ubuntu24:~/Lockbox$ make clean
make -C firmware clean
make[1]: Entering directory '/home/lilbodyn/Lockbox/firmware'
rm -rf build
make[1]: Leaving directory '/home/lilbodyn/Lockbox/firmware'
lilbodyn@Ubuntu24:~/Lockbox$ make build
docker build -t stm32c0-wokwi-build:local docker
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
Install the buildx component to build images with BuildKit:
https://docs.docker.com/go/buildx/

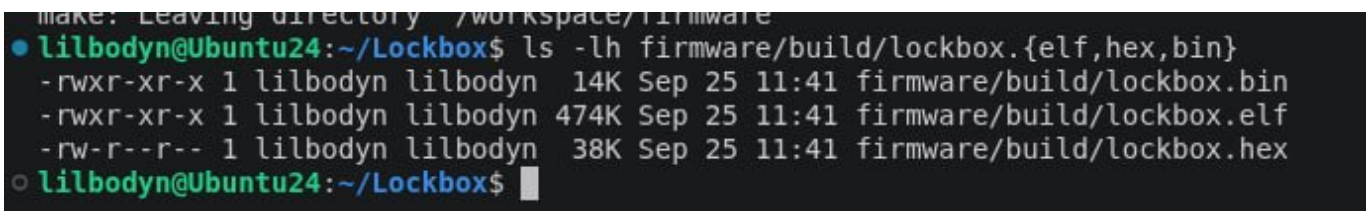
Sending build context to Docker daemon 2.048kB
Step 1/4 : FROM ubuntu:22.04
--> b8b6ee6aa931
Step 2/4 : ARG DEBIAN_FRONTEND=noninteractive
--> Using cache
--> 2a3bade6f42e
Step 3/4 : RUN apt-get update && apt-get install -y --no-install-recommends make gcc-arm-none-eabi binutils-arm-none-eabi
libnewlib-arm-none-eabi ca-certificates && rm -rf /var/lib/apt/lists/*
--> Using cache
--> a03a80407cf5
Step 4/4 : WORKDIR /workspace
--> Using cache
--> 557e5bf48a35
```

Рисунок 4 – Очищення та початок збірки в Ubuntu



```
AL_Driver/Inc -IDrivers/STM32C0xx_HAL_Driver/Inc/Legacy -IDrivers/CMSIS/Device/ST/STM32C0xx/Include -IDrivers/CMSIS/Include -Og -Wall -f
data-sections -ffunction-sections -g -gdwarf-2 -MMD -MP -MF"build/startup_stm32c031xx.d" startup_stm32c031xx.s -o build/startup_stm32c03
1xx.o
arm-none-eabi-gcc build/main.o build/stm32c0xx_it.o build/stm32c0xx_hal_msp.o build/syscalls.o build/sysmem.o build/stm32c0xx_hal.o buil
d/stm32c0xx_hal_cortex.o build/stm32c0xx_hal_rcc.o build/stm32c0xx_hal_rcc_ex.o build/stm32c0xx_hal_flash.o build/stm32c0xx_hal_flash_ex
.o build/stm32c0xx_hal_gpio.o build/stm32c0xx_hal_dma.o build/stm32c0xx_hal_dma_ex.o build/stm32c0xx_hal_pwr.o build/stm32c0xx_hal_pwr_e
x.o build/stm32c0xx_hal_exti.o build/stm32c0xx_hal_i2c.o build/stm32c0xx_hal_i2c_ex.o build/stm32c0xx_hal_uart.o build/stm32c0xx_hal_uart
ex.o build/system_stm32c0xx.o build/startup_stm32c031xx.o -mcpu=cortex-m0plus -mthumb -specs=nano.specs -TSTM32C031C6Tx_FLASH.ld -l
c -lm -lnosys -Wl,-Map=build/lockbox.map,--cref -Wl,--gc-sections -o build/lockbox.elf
arm-none-eabi-size build/lockbox.elf
text      data      bss      dec      hex filename
13460      120      1808    15388    3c1c build/lockbox.elf
arm-none-eabi-objcopy -O ihex build/lockbox.elf build/lockbox.hex
arm-none-eabi-objcopy -O binary -S build/lockbox.elf build/lockbox.bin
make: Leaving directory '/workspace/firmware'
lilbodyn@Ubuntu24:~/Lockbox$
```

Рисунок 5 – Завершення збірки без помилок



```
make: Leaving directory '/workspace/firmware'
lilbodyn@Ubuntu24:~/Lockbox$ ls -lh firmware/build/lockbox.{elf,hex,bin}
-rwxr-xr-x 1 lilbodyn lilbodyn 14K Sep 25 11:41 firmware/build/lockbox.bin
-rwxr-xr-x 1 lilbodyn lilbodyn 474K Sep 25 11:41 firmware/build/lockbox.elf
-rw-r--r-- 1 lilbodyn lilbodyn 38K Sep 25 11:41 firmware/build/lockbox.hex
lilbodyn@Ubuntu24:~/Lockbox$
```

Рисунок 6 – Три отримані формати прошивки

Окремо перевірів збірку з чистого клону гілки без каталогу `firmware/build`. Її журнал додано в коміті 44edf77. Це важливо для відтворення роботи після клонування репозиторію, а не лише в моїй робочій папці.

UART і світлодіод у симуляторі:

Функція `__io_putchar()` передає один символ через `HAL_UART_Transmit()` в USART2. Тому виклик `printf()` можна читати в UART-журналі Wokwi. Після `HAL_Init()`, налаштування годинника та периферії `main()` один раз друкує `Lockbox ready`. Далі цикл змінює стан LED на PA5, друкує `tick` з часом від `HAL_GetTick()` і чекає 1000 мс.

Файл `wokwi.toml` вказує симулятору на зібрані ELF і HEX. На рисунках 7 і 8 видно один запуск: рядки `tick` продовжуються, а LD4 переходить із увімкненого стану у вимкнений. Це підтверджує роботу UART та GPIO в симуляції. Кнопка RESET лише перезапускає прошивку; окремої логіки кнопки користувача в LP1 немає.

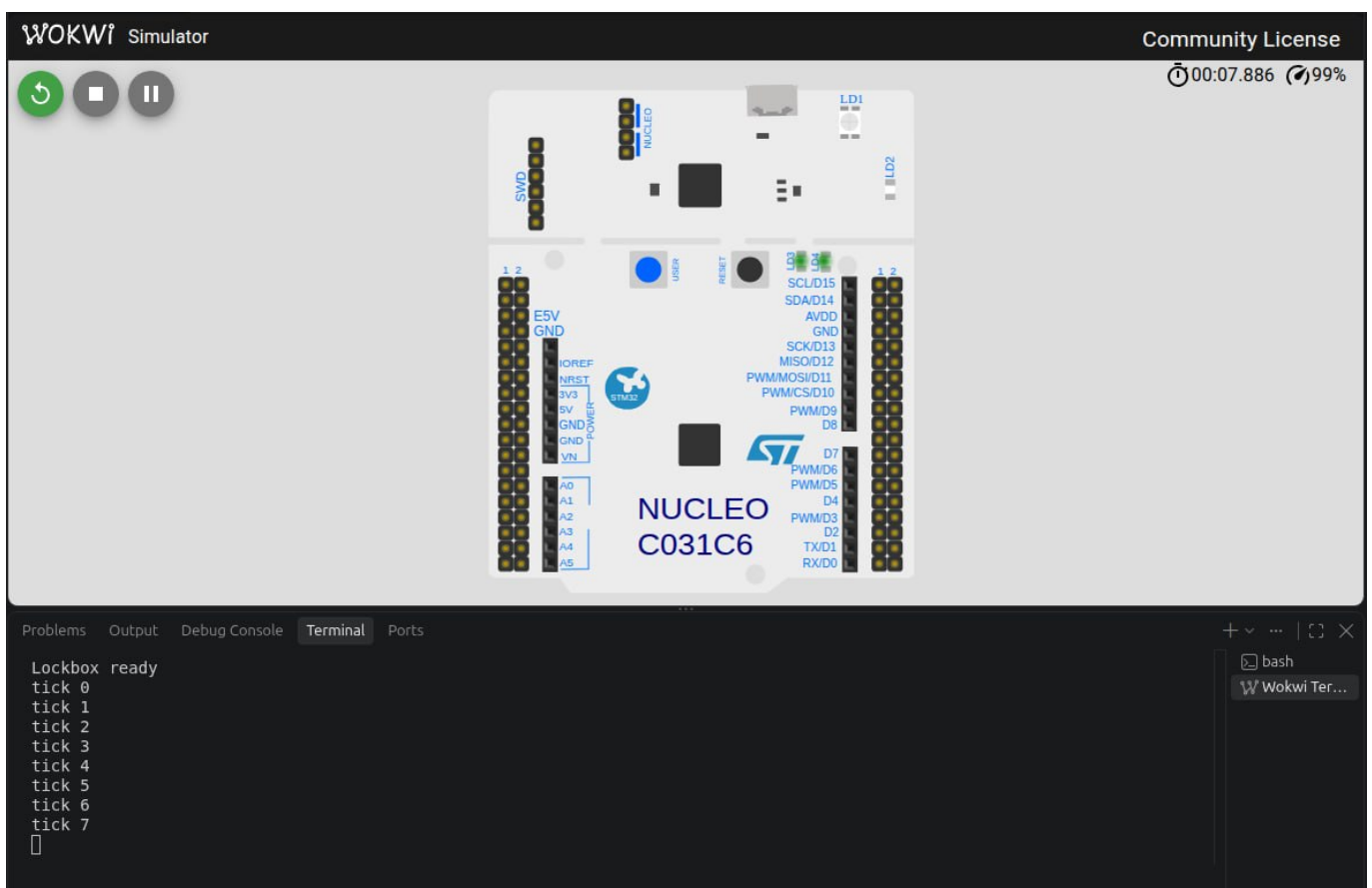


Рисунок 7 – Wokwi: UART-журнал і ввімкнений LD4

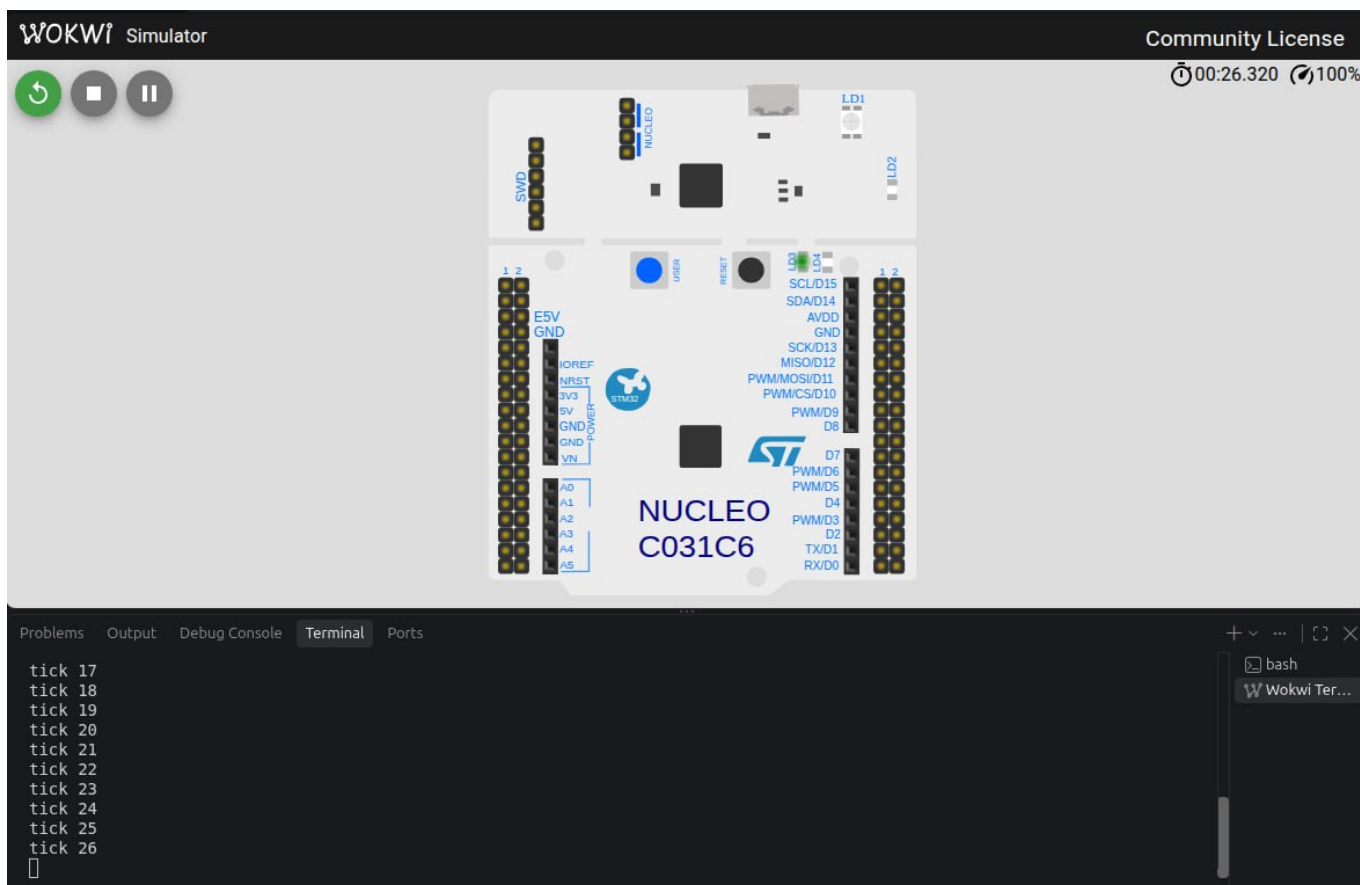


Рисунок 8 – Wokwi: наступні повідомлення та вимкнений LD4

Складнощі під час підготовки:

1. **Плата з методички та симулятор.** Для прикладу в методичці названо F411RE/F446RE, але поточна модель проекту у Wokwi board-st-nucleo-c031c6. Тому перейшов на C031C6 і відповідні HAL-файли. Вибір платформи уточнив у ea14205; погодження з викладачем лишається відкритим.
2. **Відкриття .iuc у новішому CubeMX.** Програма показала повідомлення про іншу версію пакета STM32Cube FW C0. Щоб лише переглянути наявну конфігурацію та зробити скриншоти, відкрив файл у режимі сумісності. Це локальна дія, коміту для неї немає.
3. **Підключення до Ubuntu VM.** Перші спроби SSH використовували старе ім'я користувача, тоді як у VM активний обліковий запис lilbodyn. Після уточнення адреси й імені підключення запрацювало. Це налаштування машини, без зміни репозиторію.

4. **Запуск VS Code у VM.** Спочатку команда `code` не була доступна, а після встановлення редактор не відкривав проєкт через права на локальні файли. Встановив VS Code і виправив власника та права папки. Це локальна дія, без коміту.
5. **Запуск Wokwi.** Розширення вимагало ключ для роботи в VS Code. Після введення отриманого ключа відкрив симуляцію та перевірів UART і LED, що видно на рисунках 7 і 8. Ліцензія зберігається в середовищі користувача, не в Git.

Висновки:

Мета ЛР1 досягнута в межах форми Б: прошивка збирається в Docker, Wokwi запускає її, UART показує Lockbox ready і послідовність tick, а LD4 змінює стан. Я перевірів, як налаштування USART2, GPIO й тактування в CubeMX відповідають викликам HAL у коді. Готові ELF, HEX і BIN можна відтворити після чистого клонування.

Повна охоронна логіка ще не реалізована. Наступний крок у ЛР2: підключити MPU6050, фоторезистор і PIR, написати окремі перевірки для кожного входу, а потім перевірити їх одночасну роботу. Також треба погодити C031C6 як заміну платі з методички.

Матеріали:

1. Репозиторій Lockbox: <https://github.com/lilbodyn/Lockbox>.
2. PRD і огляд компонентів: <https://github.com/lilbodyn/Lockbox/tree/feature/lab01-setup/docs>.
3. Початковий приклад для Nucleo-C031C6: <https://github.com/wokwi/stm32-hello-wokwi>.
4. Документація Wokwi для VS Code: <https://docs.wokwi.com/vscode/project-config>.